

Jenkins task 2

TASK 1: Configure 2 Slave (Agent) Machines in Jenkins Master

Objective

To configure **two Jenkins slave (agent) machines** and connect them to the **Jenkins master** using **SSH**, then verify by running jobs on the slaves.

—> Steps on Slave Machine-1 (slave-01)

Steps to be done on slave

- > Login to slave machine..
- > switch to root user
- > install jdk 11 (amazon-linux-extras install java-openjdk11)
- > Create ssh-keygen
- > cat id_rsa. pub › authorized_keys
- > chmod 700 authorized keys

Step 1: Create EC2 Instance

- Name: slave-01
- OS: Amazon Linux 2023
- Allow SSH (22)

☐	jenkins-master	i-0fe26bc99f7f8361b	Running	🔍 🔍	t3.micro	3/3 checks passed	View alarms +	us-east-1f	ec2-18-206-12-159.co...	18.206.12.15
☒	slave 1	i-0ef72a93d8a3ec60e	Running	🔍 🔍	t3.micro	Initializing	View alarms +	us-east-1f	ec2-3-236-172-31.com...	3.236.172.31

Step 2: Login & Switch to Root

```
ssh -i key.pem ec2-user@<slave-01-ip>
sudo -i
```

Step 3: Install Java

```
dnf install -y java-17-amazon-corretto
Verify:
```

```
java -version
```

```

      #_
     _###_
    _####_
   _####_
  _####_
 _####_
V~'-'>
~~~~
~~~.~.~
~~~/_/
   /m/'

Amazon Linux 2023

https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-172-31-69-152 ~]$ sudo -i
[root@ip-172-31-69-152 ~]# yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-69-152 ~]# yum install -y java-17-amazon-corretto
Last metadata expiration check: 0:01:43 ago on Wed Dec 24 11:03:01 2025.
Dependencies resolved.
=====
Package                                     Architecture
=====
Installing:
  java-17-amazon-corretto                   x86_64
Installing dependencies:
  alsa-lib                                  x86_64
  cairo                                    x86_64
  dejavu-sans-fonts                         noarch
  dejavu-sans-mono-fonts                   noarch
  dejavu-serif-fonts                       noarch
  fontconfig                               x86_64
  fonts-filesystem                         noarch
  freetype                                 x86_64
  giflib                                   x86_64
  google-noto-fonts-common                 noarch
  google-noto-sans-vf-fonts                noarch
  graphite2                                x86_64
  harfbuzz                                  x86_64
  java-17-amazon-corretto-headless         x86_64
  javapackages-filesystem                 noarch
  langpacks-core-font-en                  noarch
  libICE                                   x86_64
=====

i-0ef72a93d8a3ec60e (slave 1 )
PublicIPs: 3.236.172.31  PrivateIPs: 172.31.69.152
```

Step 4: Generate SSH Key

ssh-keygen

(Press Enter for all prompts)

```
[root@ip-172-31-69-152 ~]# ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:DD9k078f8xkAS+bGl9jwMl4EjlMywQwuogJcS+Dhe8w root@ip-172-31-69-152.ec2.internal
The key's randomart image is:
+---[RSA 3072]-----+
|  o.o  .++o         |
| + + . .  +* .      |
| .+ o ...+o.* .     |
| . = . * *.O .      |
| .o E   S  O.B       |
| . .      .o =..     |
|                   ..o. |
|                   . +o |
|                   ...  |
+----[SHA256]-----+
[root@ip-172-31-69-152 ~]# cd .ssh
[root@ip-172-31-69-152 .ssh]# ll
total 12
-rw-----. 1 root root  555 Dec 24 11:02 authorized_keys
-rw-----. 1 root root 2622 Dec 24 11:05 id_rsa
-rw-r--r--. 1 root root  588 Dec 24 11:05 id_rsa.pub
[root@ip-172-31-69-152 .ssh]#
```

i-0ef72a93d8a3ec60e (slave 1)

PublicIPs: 3.236.172.31 PrivateIPs: 172.31.69.152

Step 5: Configure Authorized Keys

cd ~/.ssh

cat id_rsa.pub > authorized_keys

chmod 700 authorized_keys

```
[root@ip-172-31-69-152 .ssh]# cat id_rsa.pub > authorized_keys
[root@ip-172-31-69-152 .ssh]# ll
total 12
-rw-----. 1 root root  588 Dec 24 11:07 authorized_keys
-rw-----. 1 root root 2622 Dec 24 11:05 id_rsa
-rw-r--r--. 1 root root  588 Dec 24 11:05 id_rsa.pub
[root@ip-172-31-69-152 .ssh]# chmod 700 authorized_keys
[root@ip-172-31-69-152 .ssh]# ll
total 12
-rwx-----. 1 root root  588 Dec 24 11:07 authorized_keys
-rw-----. 1 root root 2622 Dec 24 11:05 id_rsa
-rw-r--r--. 1 root root  588 Dec 24 11:05 id_rsa.pub
[root@ip-172-31-69-152 .ssh]#
```

Steps on Slave Machine-2 (slave-02)

Repeat same steps as slave-01:

```
ssh -i key.pem ec2-user@<slave-02-ip>
sudo -i
dnf install -y java-17-amazon-corretto
ssh-keygen
cd ~/.ssh
cat id_rsa.pub > authorized_keys
chmod 700 authorized_keys
```


—> Steps on Jenkins Master Machine

—> Login to master machine

—> switch to root user.

—> mkdir -p /var/lib/jenkins/.ssh

—> cd /var/lib/jenkins/.ssh

—> ssh-keyscan -H SLAVE-NODE- IP-OR-HOSTNAME >>/var/lib/jenkins/ .ssh/known_hosts

```
#ssh-keyscan -H 3.236.172.31 >> /var/lib/jenkins/.ssh/known_hosts
```

—> chown jenkins:jenkins known hosts

#we need to change the owner as we ran ssh-keyscan command using "root" user.
default user of Jenkins will be "jenkins"

—> chmod 700 known hosts

Step 1: Login & Switch to Root

Step 2: Prepare Jenkins SSH Directory

```
mkdir -p /var/lib/jenkins/.ssh  
cd /var/lib/jenkins/.ssh
```

```
[root@ip-172-31-65-24 jenkins]# mkdir .ssh  
[root@ip-172-31-65-24 jenkins]# cd .ssh  
[root@ip-172-31-65-24 .ssh]# ssh-keyscan -H 3.236.172.31 >>/var/lib/jenkins/ .ssh/known_hosts  
-bash: /var/lib/jenkins/: Is a directory  
[root@ip-172-31-65-24 .ssh]# ssh-keyscan -H 3.236.172.31 >> /var/lib/jenkins/.ssh/known_hosts  
  
# 3.236.172.31:22 SSH-2.0-OpenSSH_8.7  
# 3.236.172.31:22 SSH-2.0-OpenSSH_8.7  
# 3.236.172.31:22 SSH-2.0-OpenSSH_8.7  
# 3.236.172.31:22 SSH-2.0-OpenSSH_8.7  
# 3.236.172.31:22 SSH-2.0-OpenSSH_8.7  
[root@ip-172-31-65-24 .ssh]#
```

i-0fe26bc99f7f8361b (jenkins-master)

PublicIPs: 18.206.12.159 PrivateIPs: 172.31.65.24

Step 3: Add Slave-01 to Known Hosts

```
ssh-keyscan -H <slave-01-ip> >> /var/lib/jenkins/.ssh/  
known_hosts
```

```
#ssh-keyscan -H 3.236.172.31 >> /var/lib/jenkins/.ssh/known_hosts
```

```
[root@ip-172-31-65-24 .ssh]# ssh-keyscan -H 3.236.172.31 >> /var/lib/jenkins/.ssh/known_hosts  
# 3.236.172.31:22 SSH-2.0-OpenSSH_8.7  
# 3.236.172.31:22 SSH-2.0-OpenSSH_8.7  
# 3.236.172.31:22 SSH-2.0-OpenSSH_8.7  
# 3.236.172.31:22 SSH-2.0-OpenSSH_8.7  
# 3.236.172.31:22 SSH-2.0-OpenSSH_8.7  
[root@ip-172-31-65-24 .ssh]#
```

Step 4: Fix Permissions

```
chown jenkins:jenkins known_hosts  
chmod 700 known_hosts
```

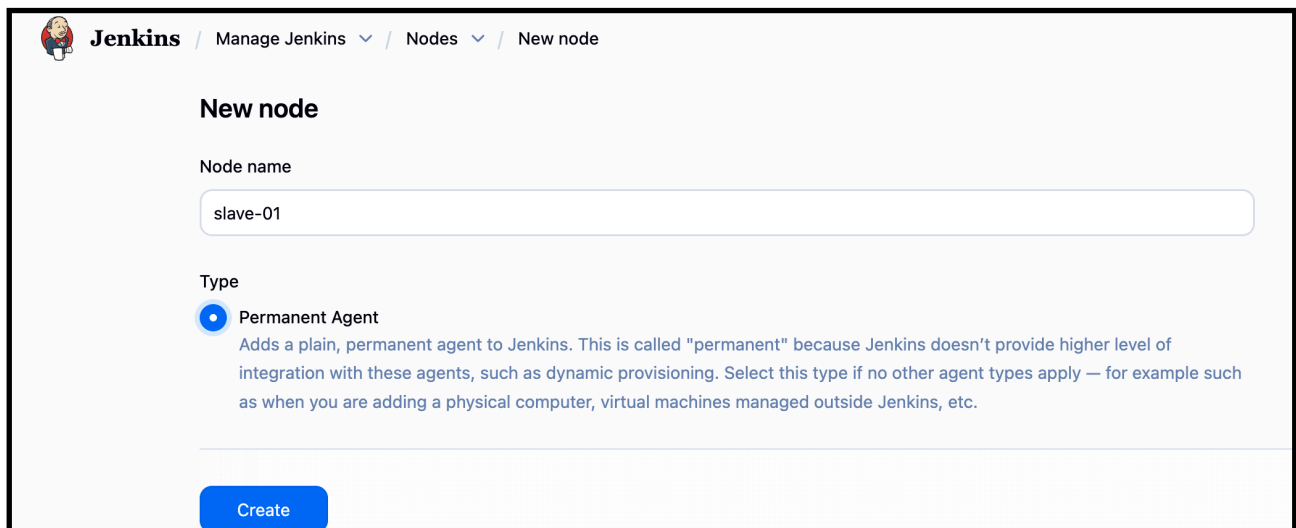
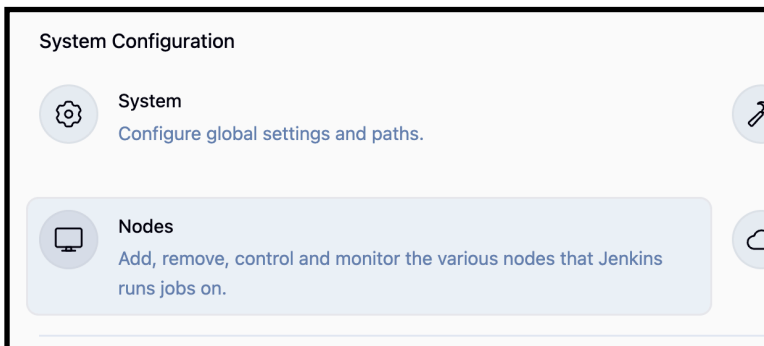
```
[root@ip-172-31-65-24 .ssh]# ll  
total 4  
-rw-r--r--. 1 root root 364 Dec 24 11:23 known_hosts  
[root@ip-172-31-65-24 .ssh]# chown jenkins:jenkins known hosts  
chown: cannot access 'known': No such file or directory  
chown: cannot access 'hosts': No such file or directory  
[root@ip-172-31-65-24 .ssh]# chown jenkins:jenkins known_hosts  
[root@ip-172-31-65-24 .ssh]# ll  
total 4  
-rw-r--r--. 1 jenkins jenkins 364 Dec 24 11:23 known_hosts  
[root@ip-172-31-65-24 .ssh]# chmod 700 known_hosts  
[root@ip-172-31-65-24 .ssh]#
```

Chown ec2-user : ec2-user known_hosts

—> Steps on Jenkins GUI

Step 1: Create New Node

- Go to: **Manage Jenkins** → **Manage Nodes and Clouds**
- Click **New Node**
- Node Name: `slave-01`
- Type: **Permanent Agent**
- Click **OK**

A screenshot of the Jenkins 'New node' configuration page. The breadcrumb trail at the top reads: 'Jenkins / Manage Jenkins / Nodes / New node'. The main heading is 'New node'. Below it, the 'Node name' field contains the text 'slave-01'. Under the 'Type' section, the 'Permanent Agent' option is selected with a radio button. A descriptive text below the radio button states: 'Adds a plain, permanent agent to Jenkins. This is called "permanent" because Jenkins doesn't provide higher level of integration with these agents, such as dynamic provisioning. Select this type if no other agent types apply — for example such as when you are adding a physical computer, virtual machines managed outside Jenkins, etc.' At the bottom of the form is a blue 'Create' button.

Step 2: Configure Node

- **Remote root directory:**
/home/ec2-user
- **Labels:**
java
- **Launch method:**
Launch agent via SSH
- **Host:**
<Slave Public IP>

Build Executor Status
(of 2 executors busy)

2

Remote root directory ?
home/ec2-user

⚠ Are you sure you want to use a relative path for the FS root? Note that relative paths require that you can assure that the selected launcher provides a consistent current working directory. Using an absolute path is highly recommended.

Labels ?
java

Usage ?
Only build jobs with label expressions matching this node

Launch method ?
Launch agents via SSH

Host ?
3.236.172.31

Credentials ?
ec2-user + Add

Host Key Verification Strategy ?
Known hosts file Verification Strategy

Advanced Edited

Step 3: Add Credentials

- Click **Add** → **Jenkins**
- Kind: SSH Username with private key
- Username: ec2-user
- Private Key: **Paste AWS .pem file content**
- Save

Launch method ?

Launch agents via SSH

Host ?

3.236.172.31

Credentials ?

- none -

+ Add

The selected credentials cannot be found

Jenkins

Host Key Verification Strategy ?

Known hosts file Verification Strategy

Advanced

Jenkins Credentials Provider: Jenkins

Add Credentials

Domain

Global credentials (unrestricted)

Kind

SSH Username with private key

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

ID ?

slave1-id

Description ?

Username

ec2-user

☐ Treat username as secret ?

Private Key

☒ Enter directly

Username

ec2-user

☐ Treat username as secret ?

Private Key

☒ Enter directly

Key

Enter New Secret Below

-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEA...
-----END RSA PRIVATE KEY-----

Passphrase

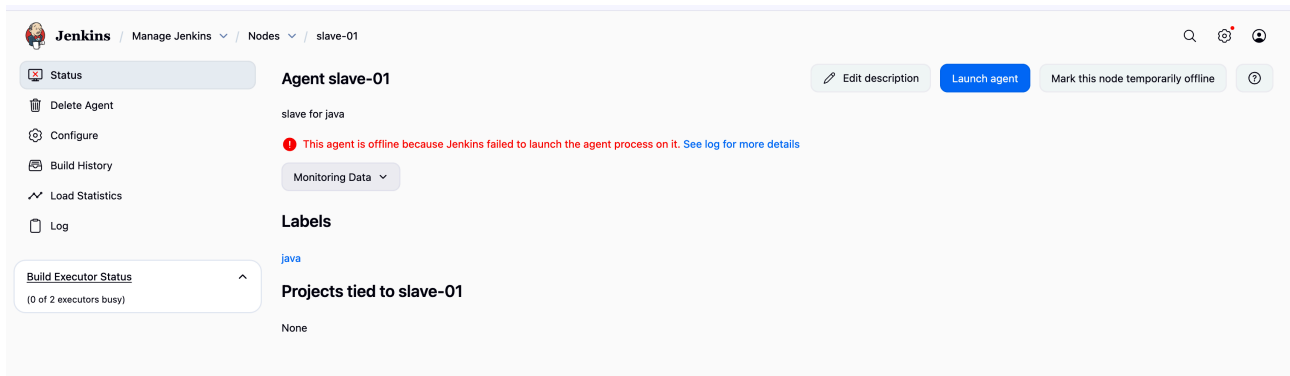
Cancel

Add

Step 4: Save & Test

- Click **Save**
- Jenkins should connect to slave via SSH
- Node status → **Online**

—> If you show issue



✗ Issue: Slave Goes Offline

Reason:

/tmp partition on slave is almost full. Jenkins automatically disables the node.

Fix: Lower Disk Threshold

- Go to **Manage Jenkins** → **Nodes**
- Click on **slave-01**
- Click **Configure**
- Under **Node Properties**:
 - Uncheck **Disk Space Monitoring Thresholds**
OR
 - **Set:**
 - Free Disk Space Threshold: 200MiB
 - Free Temp Space Threshold: 200MiB
- **Save**



Architecture ?

Clock Difference ?

Free Disk Space ?

☐ Don't mark agents temporarily offline ?

Free Space Threshold ?

200MiB

Free Space Warning Threshold ?

2GiB

Free Swap Space ?

Free Temp Space ?

☐ Don't mark agents temporarily offline ?

Free Space Threshold ?

200MiB

Free Space Warning Threshold ?

2GiB

Response Time ?

☐ Don't mark agents temporarily offline ?

Status

Delete Agent

Configure

Build History

Load Statistics

Script Console

Log

System Information

Disconnect

Agent slave-01

slave for java

Monitoring Data ▾

Labels

java

Projects tied to slave-01

None

Build Executor Status

(0 of 2 executors busy)

Jenkins

Nodes

slave-01

Log

Status

Delete Agent

Configure

Build History

Load Statistics

Script Console

Log

System Information

Disconnect

Build Executor Status

(0 of 2 executors busy)

Warning: no key algorithms provided; JENKINS-42959 disabled

SSHLauncher{host='3.236.172.31', port=22, credentialsId='slave1-id', jvmOptions='', javaPath='', prefixStartSlaveCmd='', suffixStartSlaveCmd='', launchTimeoutSeconds=60, maxNumRetries=10, retryWaitTime=15, sshHostKeyVerificationStrategy=hudson.plugins.sshslaves.verifiers.KnownHostsFileKeyVerificationStrategy, tcpNoDelay=true, trackCredentials=true}

[12/24/25 12:24:14] [SSH] Opening SSH connection to 3.236.172.31:22.

Searching for 3.236.172.31 in /var/lib/jenkins/.ssh/known_hosts

Searching for 3.236.172.31:22 in /var/lib/jenkins/.ssh/known_hosts

[12/24/25 12:24:14] [SSH] SSH host key matches key in Known Hosts file. Connection will be allowed.

[12/24/25 12:24:14] [SSH] Authentication successful.

[12/24/25 12:24:14] [SSH] The remote user's environment is:

BASH=/usr/bin/bash

BASHOPTS=checkwinsize:cmdhist:complete_fullquote:extquote:force_ignores:globasciiranges:globskipdots:hostcomplete:interactive_comments:patsub_replacement:progcomp:promptvars:sourcepath

BASHRC_SOURCED=Y

BASH_ALIASES=()

BASH_ARGC=([0]="0")

BASH_ARGV=()

BASH_CMDS=()

BASH_EXECUTION_STRING=set

BASH_LINENO=()

BASH_LOADABLES_PATH=/usr/local/lib/bash:/usr/lib/bash:/opt/local/lib/bash:/usr/pkg/lib/bash:/opt/pkg/lib/bash:.

BASH_REMATCH=()

BASH_SOURCE=()

BASH_VERSION=([0]="5" [1]="2" [2]="15" [3]="1" [4]="release" [5]="x86_64-amazon-linux-gnu")

—> check logs for conformation

Nodes

+ New Node

Configure Monitors

S	Name ↓	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-In Node	Linux (amd64)	In sync	17.45 GiB	0 B	453.91 MiB	0ms
	slave-01		N/A	N/A	N/A	N/A	N/A
	Data obtained	47 min	47 min	47 min	47 min	47 min	47 min

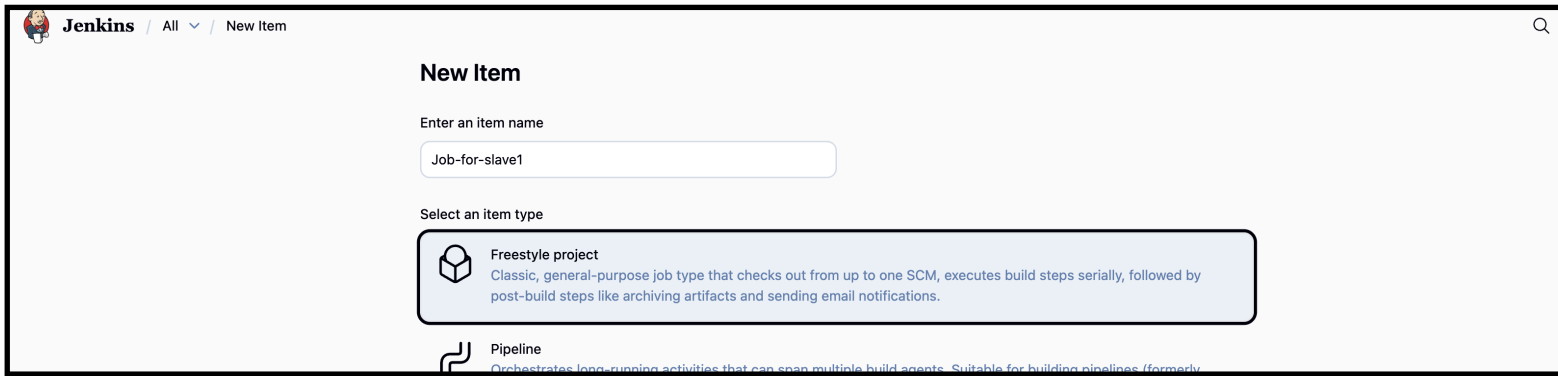
Icon: S M L

Legend

Test Slave Using Label-Based Job

Step 1: Create Freestyle Job

- Jenkins → New Item
- Job Name: Job-for-slave
- Select **Freestyle Project**
- Click **OK**



Step 2: Restrict Job to Slave

- Job Configuration → **General**
- Check **Restrict where this project can be run**
- Label Expression:

java



Step 3: Build

- Save
- Click **Build Now**
- Build runs **only on slave-01**



Slave configuration verified successfully.

Conclusion:

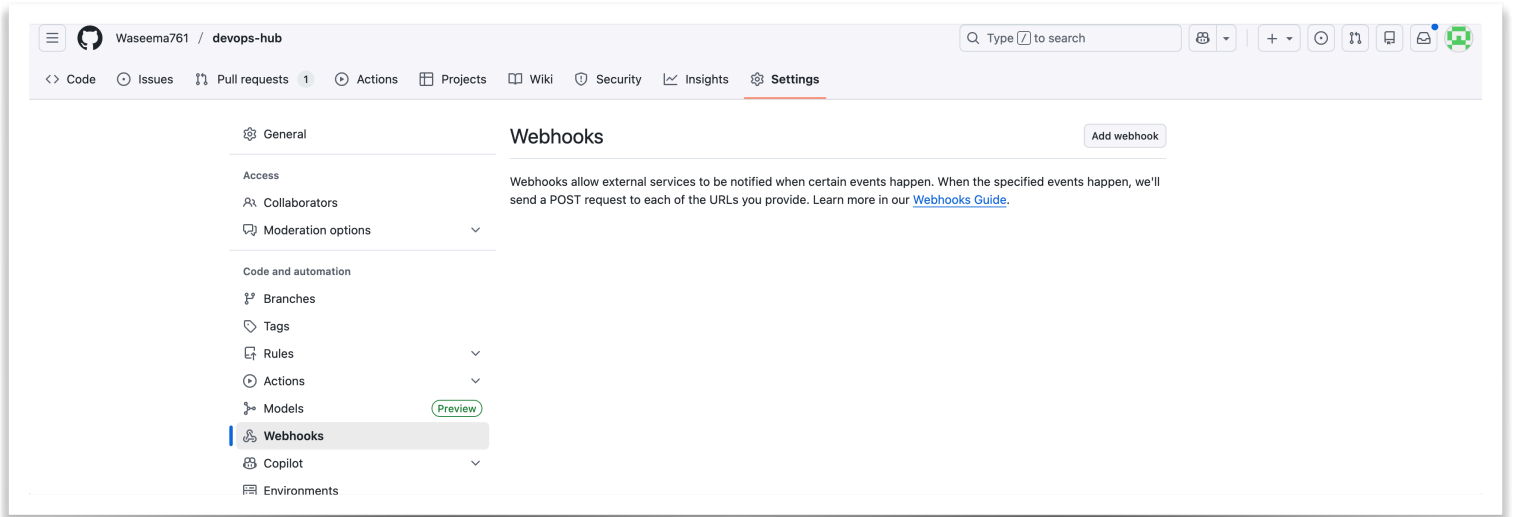
- Jobs successfully executed on specific slaves using labels

Task 2. Configure webhooks to Jenkins job.

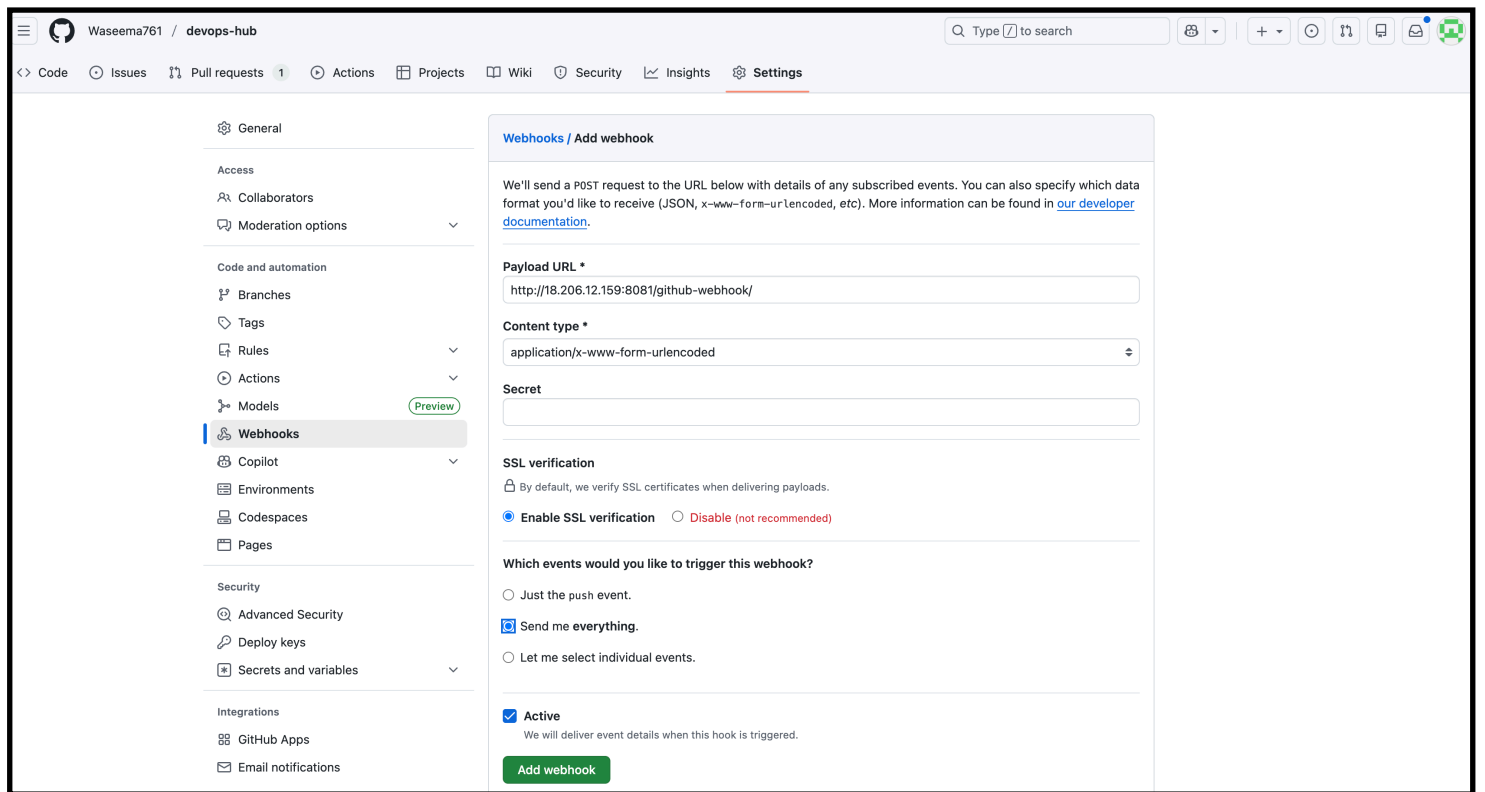
Configure GitHub Webhook

—> Configure GitHub Webhook

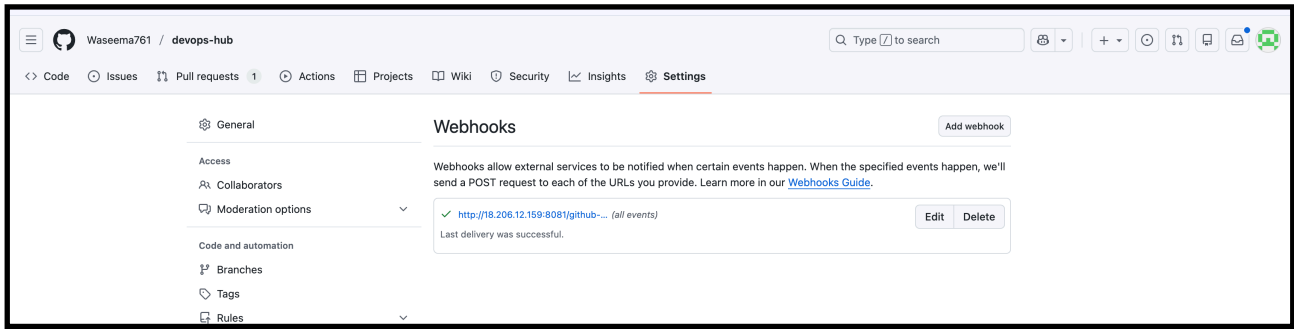
- Go to your GitHub repository → **Settings** → **Webhooks** → **Add webhook**.
- In the **Payload URL**, enter:
- <http://18.206.12.159:8081/github-webhook/>
- Click **Add webhook**.
- Choose Send me **everything**.



- In the **Payload URL**, enter:



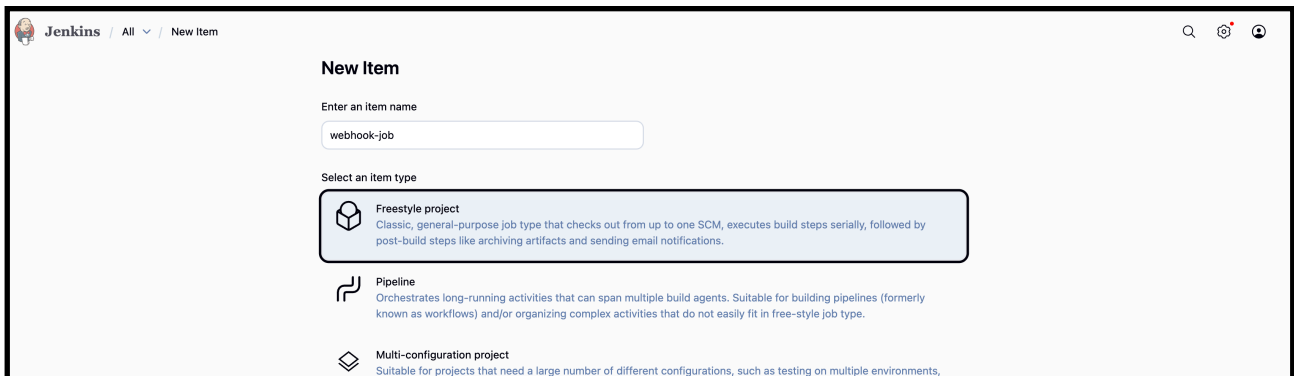
- <http://18.206.12.159:8081/github-webhook/>
- Click **Add webhook**.
- Choose Send me **everything**.



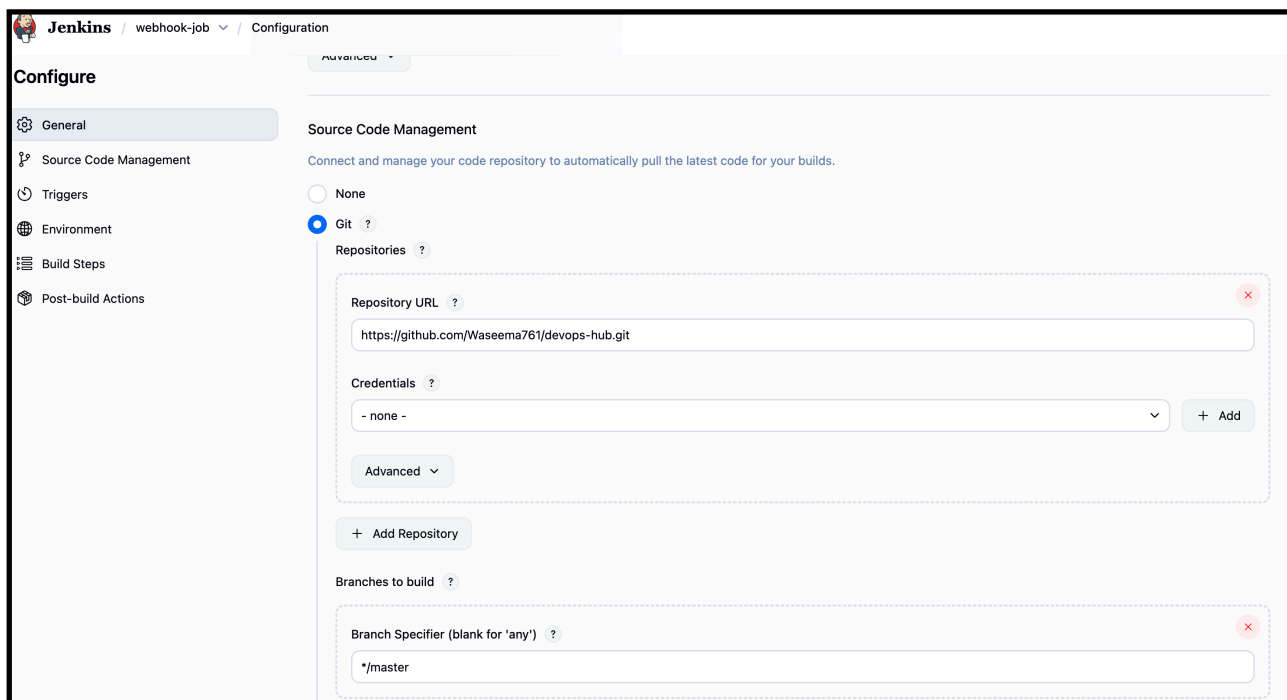
—> successfully created the webhook

Create / Configure Jenkins Job

- Create a **Freestyle Project** –(webhook-job)
- Under **Source Code Management**, choose **Git** and add your repository URL.



- Under **Source Code Management**, choose **Git** and add your repository URL.



Jenkins / webhook-job / Configuration

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ Git

Repositories

Repository URL:

Credentials: + Add

Advanced

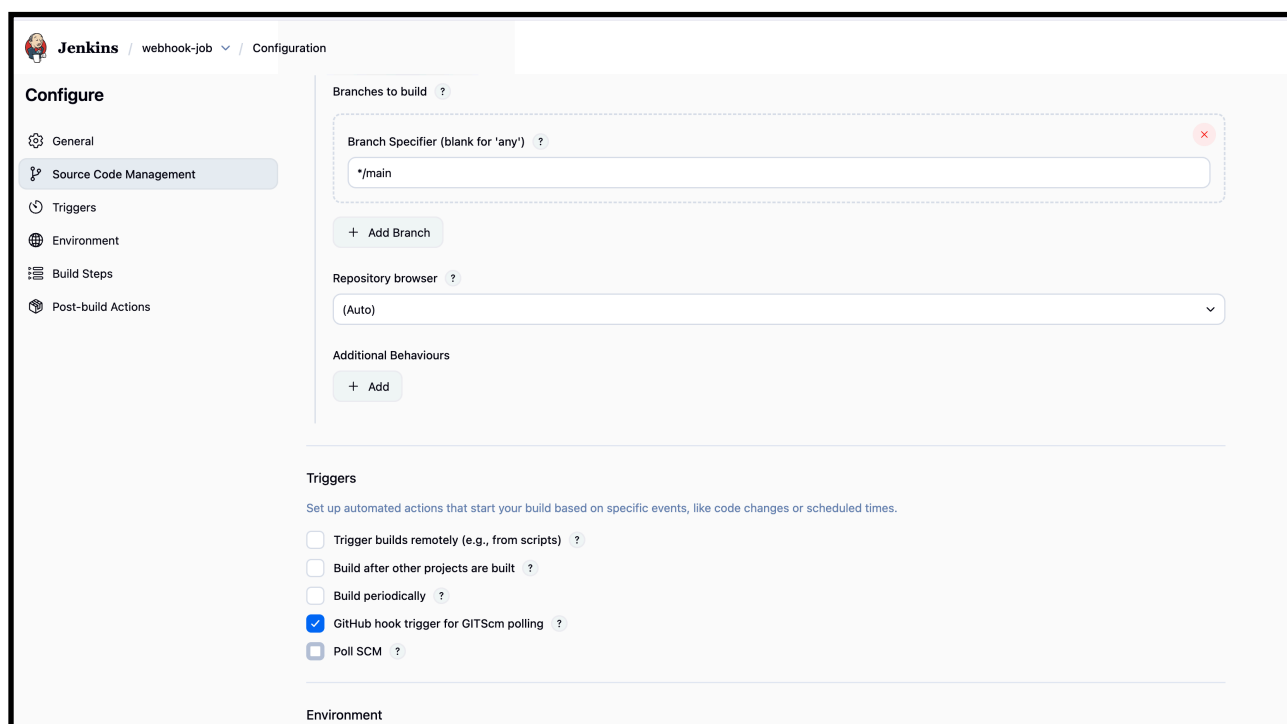
+ Add Repository

Branches to build

Branch Specifier (blank for 'any')

3. Enable Build Trigger

- In your Jenkins job → **Configure** → Scroll to **Build Triggers**.



Jenkins / webhook-job / Configuration

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Branches to build

Branch Specifier (blank for 'any')

+ Add Branch

Repository browser

Additional Behaviours + Add

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

☐ Trigger builds remotely (e.g., from scripts)

☐ Build after other projects are built

☐ Build periodically

☒ GitHub hook trigger for GITScm polling

☐ Poll SCM

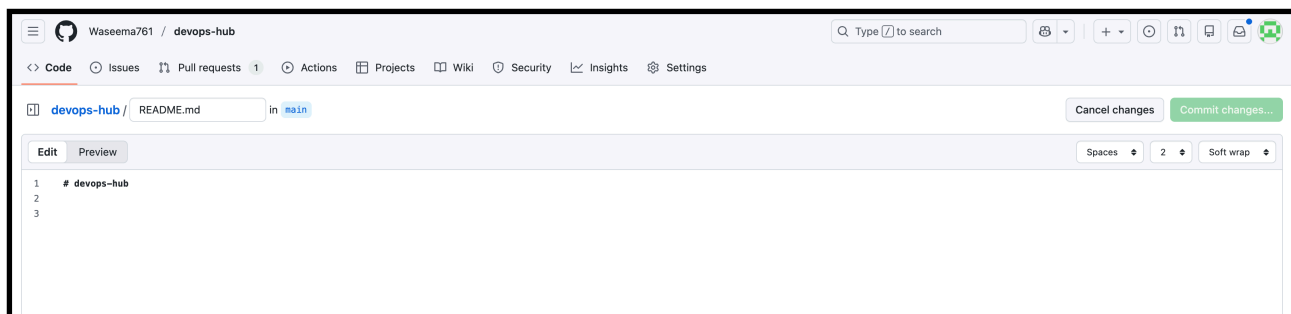
Environment

- Check:

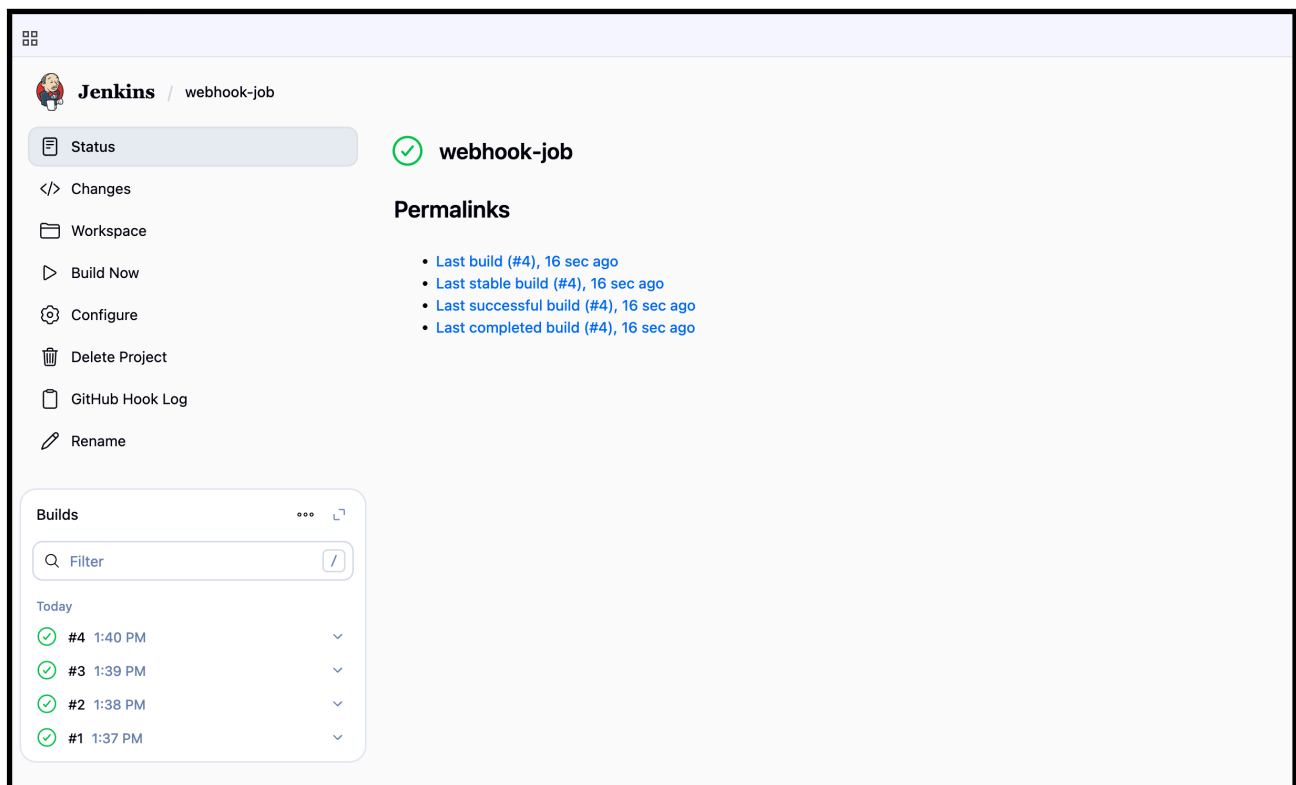
GitHub hook trigger for GITScm polling

Verify

- Push a commit to your GitHub repo.
- Jenkins job should trigger automatically.
- Check Jenkins logs to confirm webhook was received.



—> edited the readme.md file and commit the change



—-> we are able to get the build now automatically

Console Output

Download

```
Started by user waseem akram
Running as SYSTEM
Building remotely on slave-01 (java) in workspace /home/ec2-user/home/ec2-user/home/ec2-user/workspace/webhook-job
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /home/ec2-user/home/ec2-user/home/ec2-user/workspace/webhook-job/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Waseema761/devops-hub.git # timeout=10
Fetching upstream changes from https://github.com/Waseema761/devops-hub.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/Waseema761/devops-hub.git +refs/heads/*:refs/remotes/origin/* # ti
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision a82ec8b4e620fd1ae5f20d0136e675f458a32b42 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f a82ec8b4e620fd1ae5f20d0136e675f458a32b42 # timeout=10
Commit message: "Fix formatting in README.md"
> git rev-list --no-walk a82ec8b4e620fd1ae5f20d0136e675f458a32b42 # timeout=10
Finished: SUCCESS
```

Task 3 Configure poll scm and build periodical options in Jenkins job.

—> Poll SCM

This option tells Jenkins to check the source code repository (Git, SVN, etc.) at defined intervals. If it detects any change (new commit), it triggers a build.

- Go to your Jenkins job → **Configure**.
- Scroll down to **Build Triggers**.
- Check **Poll SCM**.
- In the **Schedule** box, use cron-style syntax:

Environment

Build Steps

Post-build Actions

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

☐ Trigger builds remotely (e.g., from scripts) ?

☐ Build after other projects are built ?

☐ Build periodically ?

☐ GitHub hook trigger for GITScm polling ?

☒ Poll SCM ?

Schedule ?

You appear to be missing whitespace between * and *.

☐ Ignore post-commit hooks ?

--> went to GitHub repo and commit changes

--> we will be getting the trigger build now

Jenkins / webhook-job

Status

Changes

Workspace

Build Now

Configure

Delete Project

Git Polling Log

Rename

webhook-job

Permalinks

Last build (#6), 20 sec ago

Last stable build (#6), 20 sec ago

Last successful build (#6), 20 sec ago

Last completed build (#6), 20 sec ago

Builds

Filter

Today

#6 2:08 PM

#5 1:45 PM

#4 1:40 PM

#3 1:39 PM

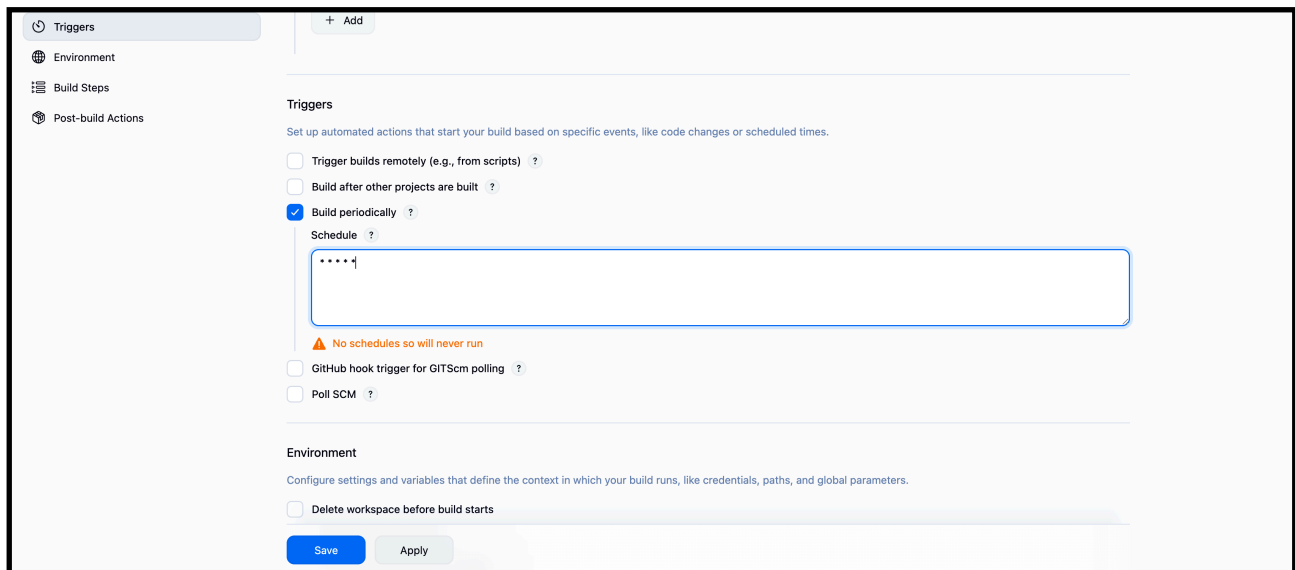
#2 1:38 PM

#1 1:37 PM

—> build periodical

This option triggers builds **whether or not there are SourceCodeManagement (scm) changes**.

- Go to your Jenkins job → **Configure**.
- Scroll down to **Build Triggers**.



The screenshot shows the Jenkins job configuration interface. On the left sidebar, the 'Triggers' tab is selected. The main content area is titled 'Triggers' and includes a sub-header: 'Set up automated actions that start your build based on specific events, like code changes or scheduled times.' There are four checkboxes: 'Trigger builds remotely (e.g., from scripts)', 'Build after other projects are built', 'Build periodically' (which is checked), and 'GitHub hook trigger for GITScm polling'. Below the 'Build periodically' checkbox is a 'Schedule' text box containing the cron expression 'H * * * *'. A warning icon and message state 'No schedules so will never run'. Below the schedule box are two more checkboxes: 'Poll SCM' and 'Delete workspace before build starts'. At the bottom of the configuration area are 'Save' and 'Apply' buttons.

- Check **Build periodically**.
- Enter a cron-style schedule in the **Schedule** box:

 Jenkins

/ webhook-job

Status

</> Changes

Workspace

Build Now

Configure

Delete Project

Rename

webhook-job

Permalinks

Last build (#7), 12 sec ago

Last stable build (#7), 12 sec ago

Last successful build (#7), 12 sec ago

Last completed build (#7), 12 sec ago

Builds

Filter

Today

#7 2:12 PM

#6 2:08 PM

#5 1:45 PM

#4 1:40 PM

#3 1:39 PM

#2 1:38 PM

#1 1:37 PM

Conclusion : we are able to trigger the build now of the job based on build periodical and poll scm

Task 4 Take backup of Jenkins server by using bash script.

Objective

To take a **manual backup of Jenkins server** using a **Bash script** by backing up Jenkins home directory.

What We Are Backing Up

Jenkins stores all important data in:

`/var/lib/jenkins`

This includes:

- Jobs
- Plugins
- Configurations
- Credentials
- Build history

—-> below script is used for creating the backup of the Jenkins server

```
#!/bin/bash

JENKINS_HOME="/var/lib/jenkins"
BACKUP_DIR="/root/jenkins-backups"
DATE=$(date +%F)
BACKUP_FILE="$BACKUP_DIR/jenkins_backup_${DATE}.tar.gz"

echo "Stopping Jenkins..."
systemctl stop jenkins

echo "Taking Jenkins backup..."
tar -czf "$BACKUP_FILE" "$JENKINS_HOME"

echo "Starting Jenkins..."
systemctl start jenkins

if [ -f "$BACKUP_FILE" ]; then
    echo "Backup successful: $BACKUP_FILE"
else
    echo "Backup failed"
    exit 1
fi

~
~
~
~
~
~
~
~
~
~
```

Step 1: Create Backup Directory

```
mkdir -p /backup/jenkins
```

Step 2: Create Backup Script

Create file:

vi backup.sh

→ add the script too Create the backup of the Jenkins server application

```
#!/bin/bash

JENKINS_HOME="/var/lib/jenkins"
BACKUP_DIR="/root/jenkins-backups"
DATE=$(date +%F)
BACKUP_FILE="$BACKUP_DIR/jenkins_backup_$DATE.tar.gz"

echo "Stopping Jenkins..."
systemctl stop jenkins

echo "Taking Jenkins backup..."
tar -czf "$BACKUP_FILE" "$JENKINS_HOME"

echo "Starting Jenkins..."
systemctl start jenkins

if [ -f "$BACKUP_FILE" ]; then
    echo "Backup successful: $BACKUP_FILE"
else
    echo "Backup failed"
    exit 1
fi
```

Step 3: Run Backup Script

`./backup.sh`

```
[root@ip-172-31-65-24 ~]# mkdir -p /root/jenkins-backups
[root@ip-172-31-65-24 ~]# ./backup.sh
^C
[root@ip-172-31-65-24 ~]# ./backup.sh
Stopping Jenkins...
Taking Jenkins backup...
tar: Removing leading `/' from member names
Starting Jenkins...
Backup successful: /root/jenkins-backups/jenkins_backup_2025-12-24.tar.gz
[root@ip-172-31-65-24 ~]# ls
backup.sh  jenkins-backups  slave_workspace
[root@ip-172-31-65-24 ~]# cd jenkins-backup
-bash: cd: jenkins-backup: No such file or directory
[root@ip-172-31-65-24 ~]# cd jenkins-backups
[root@ip-172-31-65-24 jenkins-backups]# ls
jenkins_backup_2025-12-24.tar.gz
[root@ip-172-31-65-24 jenkins-backups]#
```

Verify Backup

Conclusion :

successfully created the backup of the Jenkins server by using the bash script

Task 5

Take backup of Jenkins using thin backup plugin.

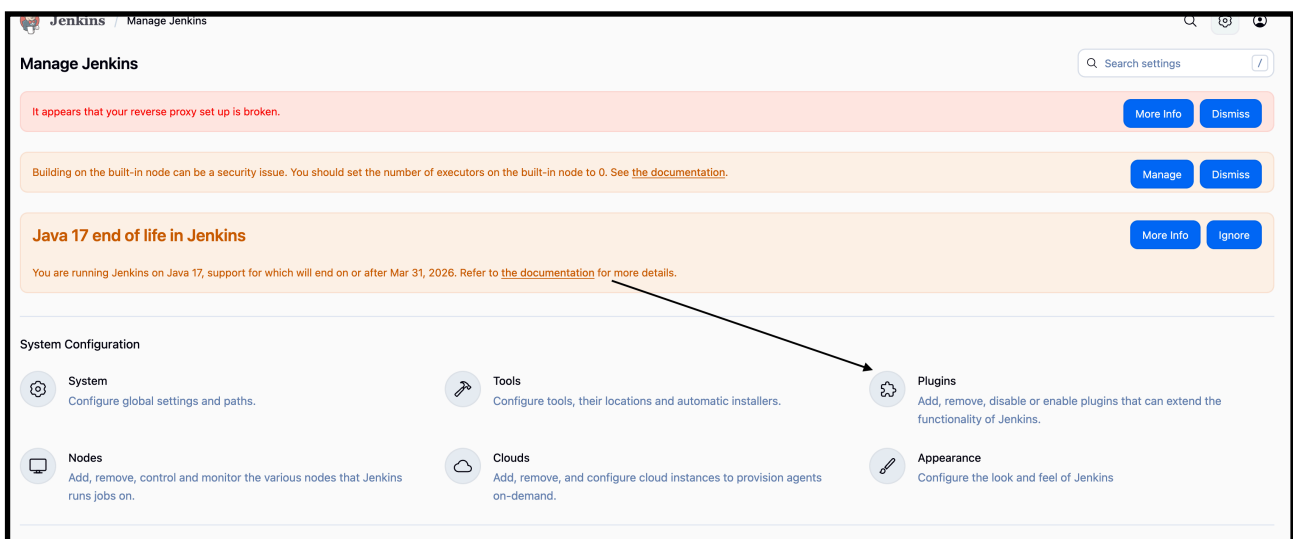
Objective

To take a **Jenkins backup using the ThinBackup plugin**, which provides an **easy and automated way** to back up Jenkins configuration, jobs, and plugins.

Steps on Jenkins GUI

Step 1: Install ThinBackup Plugin

1. Login to **Jenkins Dashboard**
2. Go to **Manage Jenkins → Manage Plugins**
3. Click on **Available Plugins**
4. Search for **ThinBackup**
5. Select **ThinBackup**
6. Click **Install**



Jenkins

Manage Jenkins

Plugins

Search

Install

Updates

Available plugins

Installed plugins

Advanced settings

thin

Install	Name	Released	Health
☐	Popper.js 2 API 2.11.6-5 Library plugins (for use by other plugins) Provides Popper.js for Jenkins Plugins. Popper can easily position tooltips, popovers or anything else with just a line of code. This plugin is deprecated. In general, this means that it is either obsolete, no longer being developed, or may no longer work. Learn more.	1 yr 5 mo ago	96
☐	AWS Credentials 254.v978a_5e206a_d7 aws Allows storing Amazon IAM credentials within the Jenkins Credentials API. Store Amazon IAM access keys (AWSAccessKeyId and AWSSecretKey) within the Jenkins Credentials API. Also support IAM Roles and IAM MFA Token.	4 mo 3 days ago	96
☐	Build Name and Description Setter 2.5.1 Build Wrappers This plug-in sets the display name and description of a build to something other than #1, #2, #3, ...	4 mo 17 days ago	100
☒	ThinBackup 2.1.3 Miscellaneous Backups the most important global and job specific configuration files.	8 mo 11 days ago	96
☐	Pipeline Maven Integration 1567.vb_2c3a_2116860 pipeline Maven This plugin provides integration with Pipeline, configures maven environment to use within a pipeline job by calling sh mvn or bat mvn. The selected maven installation will be configured and prepended to the path.	4 mo 5 days ago	100
☐	Office 365 Connector / Power Automate workflows 5.2.0 Build Notifiers		

Jenkins

Manage Jenkins

Plugins

Search

Install

Updates

Available plugins

Installed plugins

Advanced settings

Download progress

Download progress

Preparation

• Checking internet connectivity

• Checking update center connectivity

• Success

ThinBackup

Success

Loading plugin extensions

Success

Go back to the top page

(you can start using the installed plugins right away)

Restart Jenkins when installation is complete and no jobs are running

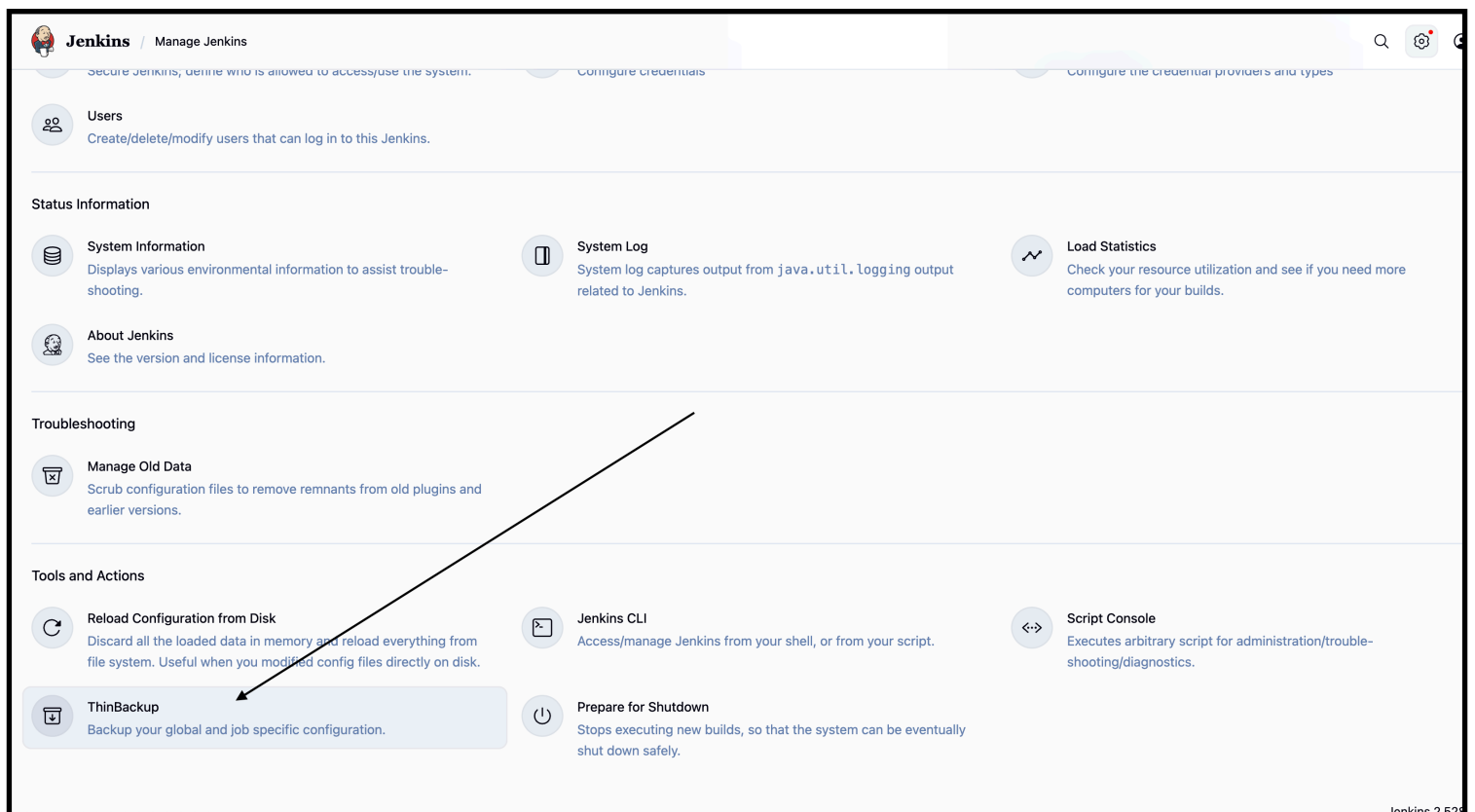
—-> thin backup successfully installed

Step 2: Configure ThinBackup

1. Go to **Manage Jenkins** → **ThinBackup**
2. Click **Settings**

Configure the following:

- **Backup Directory**



1. Set the following:
 - **Backup directory:** Path where backups will be stored (e.g., /var/Jenkins-backup).
2. Click **Save**.

ThinBackup Configuration

Backup directory ?

/var/lib/jenkins/thinbackup

Backup schedule for full backups ?

Backup schedule for differential backups ?

Max number of backup sets ?

-1

Files excluded from backup (regular expression) ?

☒ Wait until Jenkins is idle to perform a backup ?

Force Jenkins to quiet mode after specified minutes ?

120

☒ Backup build results ?

☐ Backup build archive ?

Save

Apply

Step 3: Take Manual Backup

1. Go to **Manage Jenkins** → **ThinBackup**.
2. Click **Backup Now**.
3. The backup will be created in the folder you configured (/var/Jenkins/backups-22).

 **Jenkins** / ThinBackup

+ New Item

 Build History

 Project Relationship

 Check File Fingerprint

ThinBackup

Settings are now integrated in global configuration.

 Backup now

 Restore

Build Queue ^

Build Executor Status

0/4 v

Step 4: Verify Backup

1. Navigate to the backup folder.

```
[root@ip-172-31-65-24 ~]# ls /var/lib/jenkins/thinbackup
FULL-2025-12-24_15-27
[root@ip-172-31-65-24 ~]#
```

Task 6 Setup a new Jenkins server and dump the backup taken in task4.

Objective

To set up a **new Jenkins server** and **restore Jenkins data** using the **bash backup (tar.gz)** taken in Task-4

What we already have

From Task-4, you have a backup file like:

`jenkins_backup_2025-12-24.tar.gz`

Stored in:

`/root/jenkins-backups/`

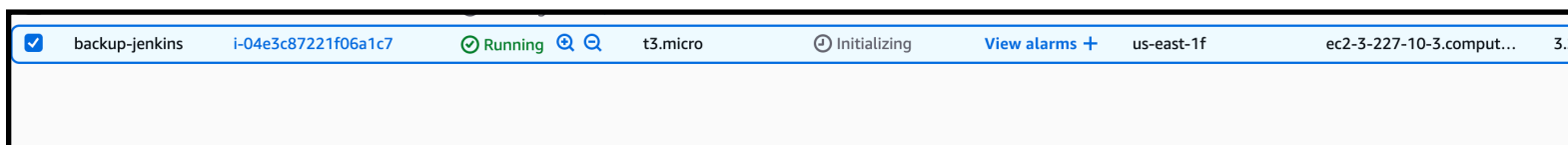
This backup contains:

/var/lib/jenkins

```
[root@ip-172-31-65-24 ~]# cd /root/jenkins-backups/  
[root@ip-172-31-65-24 jenkins-backups]# ls  
jenkins_backup_2025-12-24.tar.gz  
[root@ip-172-31-65-24 jenkins-backups]#
```

STEP-1: Launch a New Jenkins Server

1. Launch a **new EC2 instance**
 - OS: Amazon Linux
 - Open ports:
 - 22 (SSH)
 - 8080 (Jenkins)
2. Login to the new server:



—> Login to the new server:

STEP-2: Install Jenkins on New Server

Install Java

```
dnf install -y java-17-amazon-corretto
```

```
[root@ip-172-31-76-41 ~]# yum install -y java-17-amazon-corretto
Last metadata expiration check: 0:00:16 ago on Wed Dec 24 15:38:35 2025.
Dependencies resolved.
```

Package	Architecture
Installing:	
java-17-amazon-corretto	x86_64
Installing dependencies:	
alsa-lib	x86_64
cairo	x86_64
dejavu-sans-fonts	noarch
dejavu-sans-mono-fonts	noarch
dejavu-serif-fonts	noarch

1. Add Jenkins Repository

```
sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
```

```
[root@ip-172-31-76-41 ~]# sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
--2025-12-24 15:41:05-- https://pkg.jenkins.io/redhat-stable/jenkins.repo
Resolving pkg.jenkins.io (pkg.jenkins.io)... 146.75.38.133, 2a04:4e42:78::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)|146.75.38.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 192 [application/octet-stream]
Saving to: '/etc/yum.repos.d/jenkins.repo'

/etc/yum.repos.d/jenkins.repo      100%[=====]
2025-12-24 15:41:05 (9.59 MB/s) - '/etc/yum.repos.d/jenkins.repo' saved [192/192]
```

```
[root@ip-172-31-76-41 ~]# rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
[root@ip-172-31-76-41 ~]# yum install -y jenkins
Jenkins-stable
Jenkins-stable
Importing GPG key 0xEF5975CA:
  Userid      : "Jenkins Project <jenkinsci-board@googlegroups.com>"
  Fingerprint: 6366 7EE7 4BBA 1F0A 08A6 9872 5BA3 1D57 EF59 75CA
  From        : https://pkg.jenkins.io/redhat-stable/repodata/repomd.xml.key
Jenkins-stable
Dependencies resolved.
```

Package	Architecture	Version
jenkins	noarch	2.528.3-1.1

```
Transaction Summary
-----
Install 1 Package

Total download size: 91 M
Installed size: 91 M
Downloading Packages:
jenkins-2.528.3-1.1.noarch.rpm
```

2. Install Jenkins

```
sudo yum install jenkins -y
systemctl enable jenkins
systemctl start Jenkins
systemctl status jenkins
```

```
[root@ip-172-31-76-41 ~]# yum install -y jenkins
Jenkins-stable
Jenkins-stable
Importing GPG key 0xEF5975CA:
  Userid      : "Jenkins Project <jenkinsci-board@googlegroups.com>"
  Fingerprint: 6366 7EE7 4BBA 1F0A 08A6 9872 5BA3 1D57 EF59 75CA
  From        : https://pkg.jenkins.io/redhat-stable/repodata/repomd.xml
Jenkins-stable
Dependencies resolved.
=====
Package                                Architecture
=====
Installing:
jenkins                                noarch
Transaction Summary
=====
Install 1 Package
```

```
[root@ip-172-31-76-41 ~]# sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
   Active: active (running) since Wed 2025-12-24 15:44:37 UTC; 44s ago
     Main PID: 25990 (java)
       Tasks: 44 (limit: 1067)
      Memory: 345.6M
         CPU: 18.567s
    CGroup: /system.slice/jenkins.service
            └─25990 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war -

Dec 24 15:44:33 ip-172-31-76-41.ec2.internal jenkins[25990]: [LF]>
Dec 24 15:44:33 ip-172-31-76-41.ec2.internal jenkins[25990]: [LF]> *****
Dec 24 15:44:33 ip-172-31-76-41.ec2.internal jenkins[25990]: [LF]> *****
Dec 24 15:44:33 ip-172-31-76-41.ec2.internal jenkins[25990]: [LF]> *****
Dec 24 15:44:37 ip-172-31-76-41.ec2.internal jenkins[25990]: 2025-12-24 15:44:37.140+0000 [id=
Dec 24 15:44:37 ip-172-31-76-41.ec2.internal jenkins[25990]: 2025-12-24 15:44:37.158+0000 [id=
Dec 24 15:44:37 ip-172-31-76-41.ec2.internal systemd[1]: Started jenkins.service - Jenkins Con
Dec 24 15:44:37 ip-172-31-76-41.ec2.internal jenkins[25990]: 2025-12-24 15:44:37.344+0000 [id=
Dec 24 15:44:37 ip-172-31-76-41.ec2.internal jenkins[25990]: 2025-12-24 15:44:37.345+0000 [id=
Dec 24 15:44:42 ip-172-31-76-41.ec2.internal jenkins[25990]: 2025-12-24 15:44:42.209+0000 [id=
lines 1-20/20 (END)
```

Now open your master server of Jenkins

—> Step 3: Copy Your Backup to New Server

Login into your old main server you took backup

Create an test.pem file where you copy your pemkey of new-server and paste in the oldserver

—> On **old Jenkins server**: jenkins-backups —> is the folder were we placed backup file

```
scp /root/jenkins-backups/jenkins_backup_2025-12-24.tar.gz \
ec2-user@<new-server-ip>:/tmp
```

```
connection closed
[root@ip-172-31-65-24 ~]# vi test.pem
[root@ip-172-31-65-24 ~]# chmod 600 test.pem
[root@ip-172-31-65-24 ~]# scp -i test.pem /root/jenkins-backups/jenkins_backup_2025-12-24.tar.gz ec2-user@3.227.10.3:/tmp
```

```
[root@ip-172-31-65-24 ~]# vi test.pem
[root@ip-172-31-65-24 ~]# chmod 600 test.pem
[root@ip-172-31-65-24 ~]# scp -i test.pem /root/jenkins-backups/jenkins_backup_2025-12-24.tar.gz ec2-user@3.227.10.3:/tmp
jenkins_backup_2025-12-24.tar.gz
[root@ip-172-31-65-24 ~]#
```

100% 157

—>

Step 4 : Login in to new server and check the file in /tmp

jenkins_backup_2025-12-24.tar.gz —> is backup file

```
[root@ip-172-31-76-41 ~]# cd /tmp
[root@ip-172-31-76-41 tmp]# ls
hsperfdata_jenkins
jenkins_backup_2025-12-24.tar.gz
jetty-0_0_0_0-8080-war-_any-8485338969498189637
systemd-private-6b2030ca8d964696ab301c64dc65984a-chronyd.servi
systemd-private-6b2030ca8d964696ab301c64dc65984a-dbus-broker.s
[root@ip-172-31-76-41 tmp]#
```

Restore (Dump) Jenkins Backup — FINAL STEPS

Remove existing Jenkins data. Delete the contents of the new `$JENKINS_HOME` to prepare for the restore.

```
sudo rm -rf /var/lib/jenkins/*
```

Copy the file from tmp to `/var/lib/jenkins/`

```
cp /tmp/jenkins_backup_2025-12-24.tar.gz /var/lib/jenkins/
```

```
[root@ip-172-31-76-41 ~]# sudo rm -rf /var/lib/jenkins/*
[root@ip-172-31-76-41 ~]# cp /tmp/jenkins_backup_2025-12-24.tar.gz /var/lib/jenkins/
cp: -r not specified; omitting directory '/tmp/'
cp: cannot stat 'jenkins_backup_2025-12-24.tar.gz': No such file or directory
[root@ip-172-31-76-41 ~]# cp /tmp/jenkins_backup_2025-12-24.tar.gz /var/lib/jenkins/
[root@ip-172-31-76-41 ~]# cd /var/lib/jenkins
[root@ip-172-31-76-41 jenkins]# ls
jenkins_backup_2025-12-24.tar.gz
[root@ip-172-31-76-41 jenkins]#
```



STEP-2: Restore (Dump) the Backup

```
tar -xzf /tmp/jenkins_backup_2025-12-24.tar.gz -C /
```

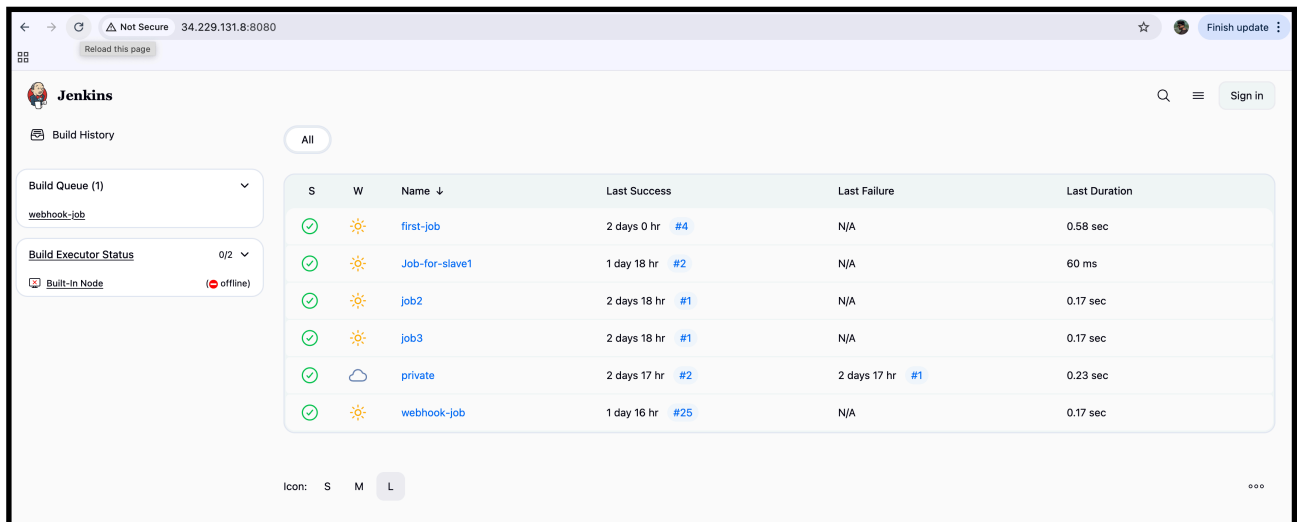
This restores:

```
var/lib/jenkins/hudson.tasks.Mailer.xml
var/lib/jenkins/hudson.triggers.SCMTrigger.xml
var/lib/jenkins/credentials.xml
var/lib/jenkins/fingerprints/
var/lib/jenkins/fingerprints/d0/
var/lib/jenkins/fingerprints/d0/96/
var/lib/jenkins/fingerprints/d0/96/04953315daa06f264186af0a41af.xml
var/lib/jenkins/fingerprints/38/
var/lib/jenkins/fingerprints/38/e9/
var/lib/jenkins/fingerprints/38/e9/f0912bde30e309e366330d6352d0.xml
var/lib/jenkins/fingerprints/8d/
var/lib/jenkins/fingerprints/8d/44/
var/lib/jenkins/fingerprints/8d/44/241c8d140043c38048ff3e051436.xml
var/lib/jenkins/.ssh/
var/lib/jenkins/.ssh/known_hosts
var/lib/jenkins/nodes/
var/lib/jenkins/nodes/slave-01/
var/lib/jenkins/nodes/slave-01/config.xml
var/lib/jenkins/queue.xml.bak
var/lib/jenkins/queue.xml
[root@ip-172-31-76-41 jenkins]#
```


configure :

```
[root@ip-172-31-76-41 jenkins]# ls -l /var/lib/jenkins/jobs/  
Job-for-slave1  
first-job  
job2  
job3  
private  
webhook-job  
[root@ip-172-31-76-41 jenkins]#
```

Below is the new server we are able to see the back up from master Jenkins

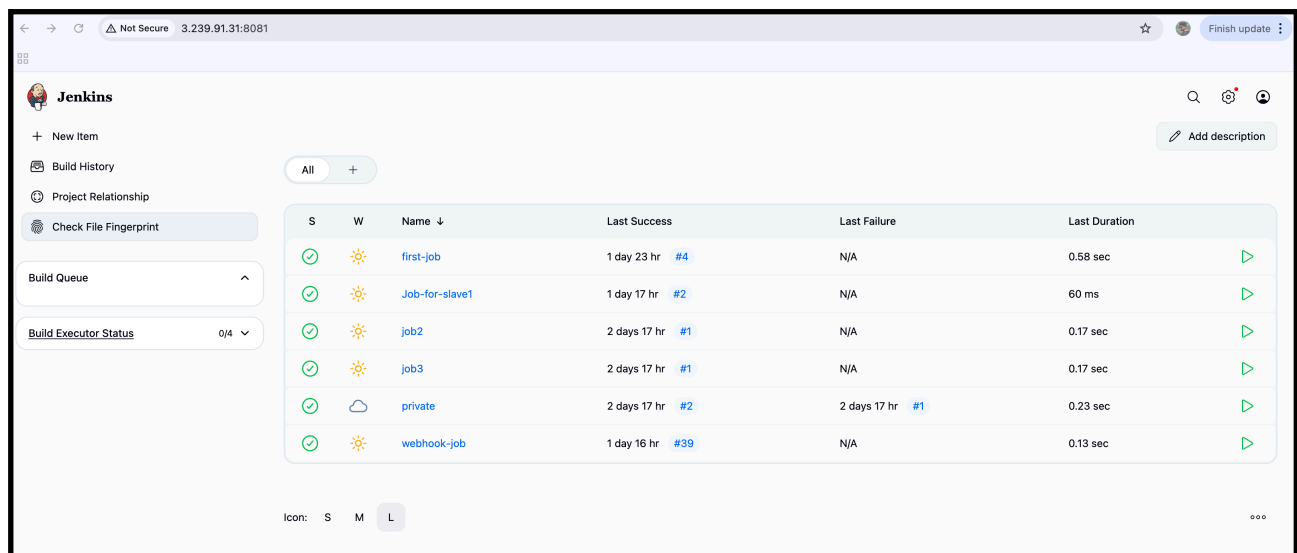


The screenshot shows the Jenkins Backup Server interface. The left sidebar contains the 'Build History' section with a 'Build Queue (1)' dropdown showing 'webhook-job' and a 'Build Executor Status' section showing '0/2' and 'Built-In Node' as 'offline'. The main table displays the build history for various jobs.

S	W	Name ↓	Last Success	Last Failure	Last Duration
✓	☀	first-job	2 days 0 hr #4	N/A	0.58 sec
✓	☀	Job-for-slave1	1 day 18 hr #2	N/A	60 ms
✓	☀	job2	2 days 18 hr #1	N/A	0.17 sec
✓	☀	job3	2 days 18 hr #1	N/A	0.17 sec
✓	☁	private	2 days 17 hr #2	2 days 17 hr #1	0.23 sec
✓	☀	webhook-job	1 day 16 hr #25	N/A	0.17 sec

Icon: S M L

Below is the master Jenkins we are able to see same data in backup Jenkins and master Jenkins



The screenshot shows the Jenkins Master Server interface. The left sidebar contains the 'Build History' section with a 'Build Queue' dropdown showing '0/4' and a 'Build Executor Status' section showing '0/4'. The main table displays the build history for various jobs, including a 'Add description' button for each row.

S	W	Name ↓	Last Success	Last Failure	Last Duration
✓	☀	first-job	1 day 23 hr #4	N/A	0.58 sec
✓	☀	Job-for-slave1	1 day 17 hr #2	N/A	60 ms
✓	☀	job2	2 days 17 hr #1	N/A	0.17 sec
✓	☀	job3	2 days 17 hr #1	N/A	0.17 sec
✓	☁	private	2 days 17 hr #2	2 days 17 hr #1	0.23 sec
✓	☀	webhook-job	1 day 16 hr #39	N/A	0.13 sec

Icon: S M L